



東京大学 ブロックチェーン
イノベーション寄付講座

ADVANCED CRYPTOGRAPHY PROGRAM • WEEK 3

zkSNARK part 1

有限体 • 多項式 • Commitment • Arithmetization

nunocloth

目次

- zkSNARK の導入
- 導入例
- 数の準備
- 多項式の準備
- 楕円曲線/離散対数
- 対話 ZK/FS
- ペアリング
- Commitment
- Arithmetization
- まとめ・補足

zkSNARK で実現したいこと

秘密を手元に置いたまま、「条件を満たすものを知っている」という主張だけを相手に納得させたい。zkSNARK は、この要求に応える証明の仕組みである。

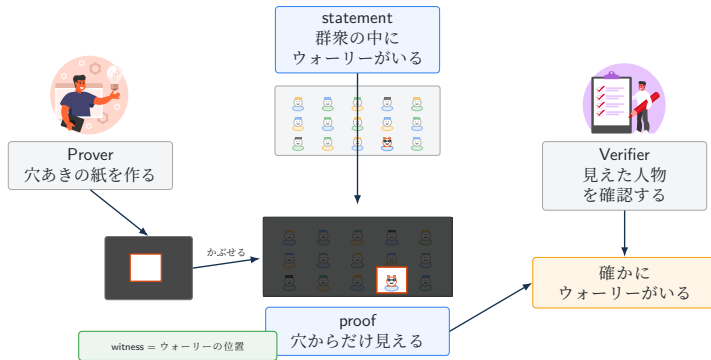
たとえばこんな場面

- パスワードを伏せたまま、正しいパスワードを知っていることを示す。
- 取引の中身を伏せたまま、ルールどおりの取引であることを示す (ブロックチェーンでの主な用途)。

登場人物は 2 人。示す側が証明者 (prover)、確かめる側が検証者 (verifier)。まず 2 つの例で感覚をつかみ、そのあと用語と性質を整理する。

例 1: ウォーリーを探せ

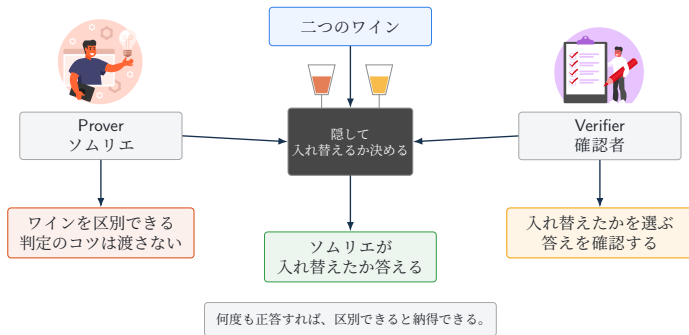
「群衆の絵のどこかにウォーリーがいる」と示したい。位置は秘密のまま。大きな紙で絵全体を覆い、ウォーリーだけが見える穴から確認してもらう。



検証者が得るのは「確かにいる」という事実だけで、絵のどこにいたかは分からないまま。

例 2: ワインソムリエ

「2つのワインを飲み分けられる」と示したい。見分け方のコツは秘密のまま。検証者が隠して入れ替えるか決め、ソムリエが当てる。



1 回の正答は偶然でも確率 $1/2$ 。10 回連続なら偶然の確率は $1/2^{10} < 0.1\%$ で、繰り返しが納得を作る。

例から用語を取り出す

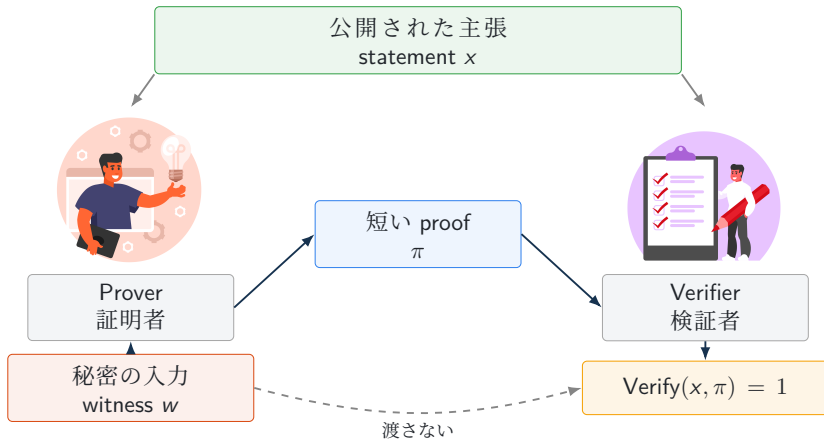
2つの例に共通する骨組みへ名前を付ける。

用語	ウォーリー	ソムリエ
statement(公開の主張)	群衆の中にウォーリーがいる	2つのワインを区別できる
witness(秘密)	ウォーリーの位置	味を見分けるコツ
proof(証拠)	穴から見える姿	当てっこへの正答の列

proof を作って示す側が prover(証明者)、proof を確かめる側が verifier(検証者)。どちらの例でも、witness は prover の手元に残ったまま検証が終わる。

登場人物と流れ

骨組みを一般の形で描くと下の図になる。以降、statement を x 、witness を w 、proof を π と書く。



statement と witness を式で見る

ここからは主張を数式で扱う。

例: 秘密の解を知っている

公開値 $y = 15$ に対して、 $w^3 + w + 5 = y$ を満たす秘密の w を知っている、と主張したい。

statement: 公開される存在主張

$$\exists w : w^3 + w + 5 = 15$$

witness: 主張を支える秘密

$$w = 2$$

verifier へ渡るのは statement と proof だけで、解 $w = 2$ は prover の手元に残る。

主張を検査プログラム R で書く

主張の「条件」の部分は、公開入力 y と秘密の候補 w を受け取って合否を返す関係 R としてまとめられる。

```
def R(y, w):  
    # y: public value  
    # w: secret candidate  
    if w**3 + w + 5 == y:  
        return 1  
    return 0
```

$y = 15$, $w = 2$ なら $2^3 + 2 + 5 = 15$ なので $R(15, 2) = 1$ 。この記法で、statement は一般に

$$\exists w \text{ such that } R(y, w) = 1$$

という形の存在主張になる。zkSNARK が証明するのはこの形の主張である。

zkSNARK とは: 名前に性質が書いてある

zkSNARK は、次の性質を同時に満たす証明システムである。頭文字がそのまま性質の一覧になっている。

- **Completeness:** 正しい witness を持つ prover の proof は受理される。
- **Knowledge soundness:** 有効な proof を作れる prover は、対応する witness を知っているはずである。
- **Zero-knowledge:** proof から witness 由来の余計な情報が漏れない。ウォーリーの例の「位置は分からないまま」に当たる。
- **Succinctness:** proof が短く、verifier は速く検証できる。
- **Non-interactive:** proof1 通で完結する。ソムリエの例で必要だった何往復もの当てっこを 1 通に置き換える。

名前の分解

zk	Zero-Knowledge
S	Succinct
N	Non-interactive
ARK	ARgument of Knowledge

簡単に言うと

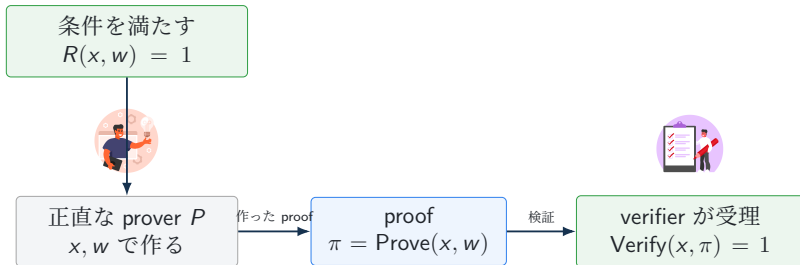
秘密を隠したまま、条件を満たすことだけを短い proof で示す。

続く 3 枚で、Completeness・Knowledge soundness・Zero-knowledge を図と式で確かめる。

定式化 1: Completeness

Completeness は、正しい statement と witness から正直に作った proof を、verifier が受理することを要求する。

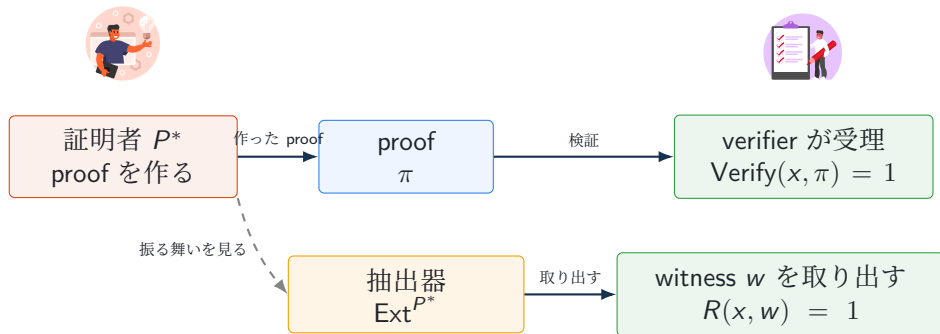
$$R(x, w) = 1 \implies \text{Verify}(x, \text{Prove}(x, w)) = 1$$



ここで x は公開された statement であり、 w は statement を正しくする witness である。この性質は、本当に条件を満たしている prover が、プロトコルのせいで落とされないことを保証する。

定式化 2: Knowledge soundness

Knowledge soundness は、有効な proof を作れるなら対応する witness を知っているはずだ、という性質である。



証明者 P^* が verifier を十分な確率で受理させられるなら、 P^* の振る舞いにアクセスする抽出器 Ext^{P^*} が $R(x, w) = 1$ となる witness w を取り出せる。

定式化 3: Zero-knowledge

zero-knowledge は、「本物の実行」と「シミュレーションで作った見かけの実行」を区別できないことを要求する。

$$\text{Real}_{P,V^*}(x, w) \approx \text{Sim}_{S,V^*}(x)$$

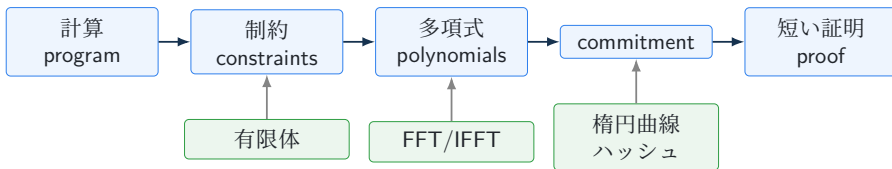
$\text{Real}_{P,V^*}(x, w)$ は、prover が witness w を使って作る本物の実行記録を表す。 $\text{Sim}_{S,V^*}(x)$ は、simulator が公開情報 x とシミュレーション用の設定情報から作る見かけの実行記録を表す。zero-knowledge では、verifier はこの二つの記録を見分けられない。

区別できなさの強さには、次の三つの段階がある。

- Perfect は、確率分布が完全に一致していることを意味する。
- Statistical は、確率分布が十分に近いことを意味する。
- Computational は、効率的に区別するアルゴリズムが存在しないことを意味する。

使う道具と講義の地図

zkSNARK は、計算の正しさの主張を段階的に変換して、短い proof へ落とし込む。



- **有限体**: すべての計算をこの中で行う。次のセクションで作り方から確認する。
- **多項式**: 多数の制約を少ない本数の等式へまとめる道具。係数と評価値の行き来を FFT が支える。
- **Commitment**: 多項式や値を短い値に固定し、あとからの書き換えを防ぐ。hash や楕円曲線を使う。

なぜ数の世界を作り直すのか

verifier が prover の計算を毎回再実行すると、検証の負担が大きくなる。そこで計算を

足し算と掛け算の等式

の集まりへ変換し、等式として検査する。

実数や浮動小数点数では丸め誤差が入り、「等しい」の判定が揺れる。等式を厳密に扱うために、有限体という閉じた数の世界を使う。

プログラムの条件 $\longrightarrow$ 有限体上の等式

有限体で嬉しいこと

- 丸め誤差がない。
- 計算結果が必ず同じ集合へ戻る。
- 多項式の議論と相性がよい。

余りの世界で計算する

整数を7で割った余りだけを見る世界を考える。

$$\{0, 1, 2, 3, 4, 5, 6\}$$

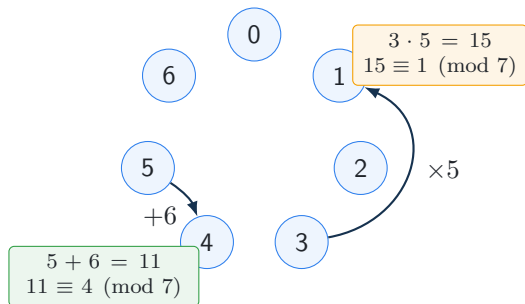
計算結果が7以上になったら、7で割った余りへ戻す。「余りが等しい」ことを $\equiv$ で書く。

$$5 + 6 = 11 \equiv 4 \pmod{7}$$

$$3 \cdot 5 = 15 \equiv 1 \pmod{7}$$

足し算・引き算・掛け算は、この規則だけでいつでも計算できる。

この集合を $\mathbb{Z}/7\mathbb{Z}$ と書く。一般に、 n で割った余りの集合が $\mathbb{Z}/n\mathbb{Z}$ 。



割り算には素数の法が要る

$\mathbb{Z}/6\mathbb{Z}$ では、0 以外の値どうしの掛け算が 0 になることがある。

$$2 \cdot 3 \equiv 0 \pmod{6}$$

このとき 2 には「掛けて 1 になる相手」がない。2x を 6 で割った余りは 0, 2, 4 のどれかで、1 には届かないため。割り算や方程式変形がここで止まる。

法を素数 p にすると

0 以外のすべての値に、掛けて 1 になる相手 (逆元) が存在する。四則演算が自由にできる数の世界を体と呼び、

$$\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$$

を有限体と呼ぶ。zkSNARK では、大きな素数 p の $\mathbb{F}_p$ の中ですべてを計算する。

逆元と割り算

a の逆元 a^{-1} は、掛けると 1 になる値。割り算は、逆元を掛ける操作として読む。

$$a \neq 0, \quad a \cdot a^{-1} = 1$$

a	1	2	3	4	5	6
a^{-1}	1	4	5	2	3	6

$\mathbb{F}_7$ の逆元表。例: $2 \cdot 4 = 8 \equiv 1 \pmod{7}$ 。

例: $\mathbb{F}_7$ で $3x = 5$ を解く

表から $3^{-1} = 5$ 。両辺に 5 を掛けて $x = 5 \cdot 5 = 25 \equiv 4 \pmod{7}$ 。検算: $3 \cdot 4 = 12 \equiv 5 \pmod{7}$ 。

なお、0 には逆元がない。有限体でも 0 割りは定義できない。

拡張 Euclid 互除法で逆元を求める

素数 p と $0 < x < p$ は互いに素なので、 $ax + bp = 1$ を満たす整数 a, b が見つかる。 $\text{mod } p$ では $bp \equiv 0$ なので $ax \equiv 1 \pmod{p}$ 、つまり a が x の逆元になる。

例: $\mathbb{F}_{13}$ で 5^{-1} を求める

余りを取りながら 1 まで下りる。

$$13 = 2 \cdot 5 + 3, \quad 5 = 1 \cdot 3 + 2, \quad 3 = 1 \cdot 2 + 1$$

下の式から順に、1 を 13 と 5 の組み合わせへ書き直す。

$$\begin{aligned} 1 &= 3 - 1 \cdot 2 \\ &= 3 - 1 \cdot (5 - 1 \cdot 3) = 2 \cdot 3 - 1 \cdot 5 \\ &= 2 \cdot (13 - 2 \cdot 5) - 1 \cdot 5 = 2 \cdot 13 - 5 \cdot 5 \end{aligned}$$

$\text{mod } 13$ で見ると $-5 \cdot 5 \equiv 1$ 。よって $5^{-1} = -5 \equiv 8$ 。検算: $5 \cdot 8 = 40 \equiv 1 \pmod{13}$ 。

巡回群と位数

集合と演算の組で、(1) 演算しても外へ出ない、(2) 単位元がある、(3) 各要素に逆元がある、(4) 結合法則が成り立つ、というものを群と呼ぶ。 $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$ は掛け算について群になる。

ある元 g を掛け続けて全体を巡れるとき、 g を生成元と呼ぶ。

例: $\mathbb{F}_7^*$ は $g = 3$ で巡れる

$$3^1 = 3, \quad 3^2 = 2, \quad 3^3 = 6, \quad 3^4 = 4, \quad 3^5 = 5, \quad 3^6 = 1 \pmod{7}$$

6 ステップで 0 以外の全要素が一度ずつ現れ、1 へ戻る。

- 群の位数: 群に含まれる要素の数。 $|\mathbb{F}_p^*| = p - 1$ 。
- 元 g の位数: $g^n = 1$ となる最小の正整数 n 。上の例では 3 の位数は 6。

のちにフーリエ変換で使う「位数 n の 1 の根」は、この位数の言葉で定義される。

なぜ多項式が出てくるのか

やりたいこと

たくさんの値や等式を、短い検査へ圧縮したい。

そのまま見る

- すべての行を確認する。
- データが大きいと検証の負担が増える。
- verifier が計算を再実行する方向へ戻ってしまう。

多項式で見る

- 値の列を多項式として表す。
- 多項式の性質でまとめて検査する。
- ランダム点で嘘を見抜きやすい。

多項式は値の列を扱う言語

多項式

$$f(X) = 3X^2 + 2X + 5$$

は二つの見方を持つ。

係数表現

$(5, 2, 3)$

定数項から順に持つ。

評価表現

$(f(x_0), f(x_1), \dots)$

点での値の列として持つ。

重要事実

次数 d 以下の多項式は、異なる $d + 1$ 点での値が決まれば一意に決まる。係数という未知数が $d + 1$ 個あり、1 点の値が 1 本の条件になる、と数えると覚えやすい。

補間: 点から式を復元する

1 次式 $f(X) = aX + b$ は 2 点で決まる。

$$f(1) = 3, \quad f(2) = 5$$

なら、傾きは $a = \frac{5-3}{2-1} = 2$ 、切片は $b = 3 - 2 \cdot 1 = 1$ なので、

$$f(X) = 2X + 1.$$

評価表現から係数表現へ戻すこの操作を補間と呼ぶ。一般の点数で同じことをする道具が、次の Lagrange 補間である。

Lagrange 補間の式

異なる点 $x_0, \dots, x_d$ と値 $y_0, \dots, y_d$ があるとする。まず、Lagrange 基底 L_i は評価点で次の振る舞いをするように作る。

$$L_i(x_j) = \begin{cases} 1 & (i = j), \\ 0 & (i \neq j). \end{cases}$$

この性質があれば、 $f(x_i) = y_i$ を満たす多項式を

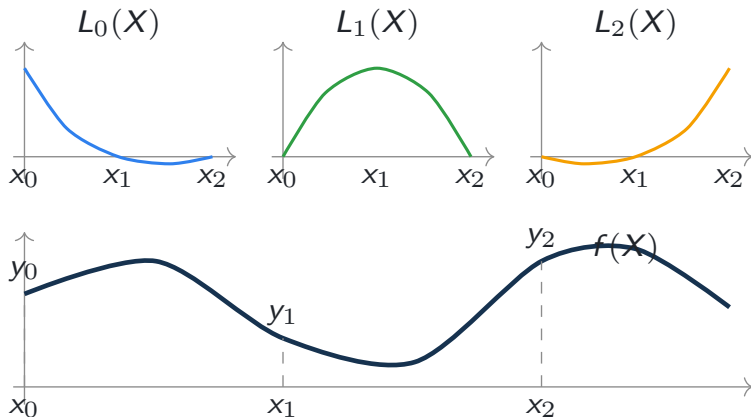
$$f(X) = \sum_{i=0}^d y_i L_i(X)$$

と書ける。
ちなみに、各 $L_i(X)$ は具体的には

$$L_i(X) = \prod_{\substack{0 \leq j \leq d \\ j \neq i}} \frac{X - x_j}{x_i - x_j}$$

と書ける。有限体では $x_i - x_j \neq 0$ なら逆元で割れる。

Lagrange 基底で $f(X)$ を作る



$$f(X) = y_0 L_0(X) + y_1 L_1(X) + y_2 L_2(X)$$

小例: 3 点から 2 次式を復元する

3 点 $(0, 1)$, $(1, 2)$, $(2, 5)$ を通る 2 次以下の多項式を、Lagrange 補間で作る。まず各基底を計算する。分母は「自分の点と他の点との差」の積。

$$L_0(X) = \frac{(X-1)(X-2)}{(0-1)(0-2)} = \frac{(X-1)(X-2)}{2}, \quad L_1(X) = \frac{X(X-2)}{(1-0)(1-2)} = -X(X-2)$$

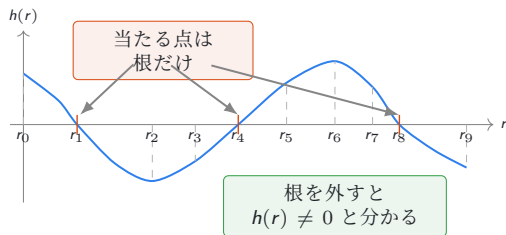
$$L_2(X) = \frac{X(X-1)}{(2-0)(2-1)} = \frac{X(X-1)}{2}$$

値 y_i を重みにして足し合わせる。

$$f(X) = 1 \cdot L_0(X) + 2 \cdot L_1(X) + 5 \cdot L_2(X) = X^2 + 1$$

検算: $f(0) = 1$, $f(1) = 2$, $f(2) = 5$ 。同じ手順が次数 d 以下 $\cdot d+1$ 点でそのまま使え、有限体では割り算を逆元で行う。

Schwartz-Zippel の補題

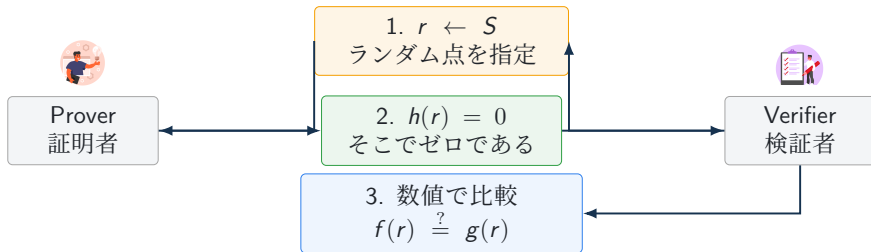


二つの多項式 f, g が等しいかを調べるために、差 $h(X) = f(X) - g(X)$ を見る。

補題の形

$$h \neq 0 \implies \Pr_{r \leftarrow S}[h(r) = 0] \leq \frac{\deg(h)}{|S|}$$

非ゼロ多項式の根は高々次数個なので、集合 S が大きいほどランダム点が根に当たりにくい。



フーリエ変換とは

フーリエ変換は、同じ対象を二つの表し方で行き来する変換として読む。多項式では、係数標識と評価標識のどちらで見てもよい。

表し方	何を持つか	読み方
係数標識	$a = (a_0, \dots, a_{n-1})$	$f(X) = \sum_{i=0}^{n-1} a_i X^i$ として見る。
評価標識	$y = (f(1), f(\omega), \dots, f(\omega^{n-1}))$	決まった点での値の列として見る。

有限体 $\mathbb{F}_p$ の中で、ある元 ω が

$$\omega^n = 1, \quad \omega^0, \omega^1, \dots, \omega^{n-1} \text{ がすべて異なる}$$

を満たすとき、 ω を位数 n の 1 の根と呼ぶ。

$$H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

を規則的な評価点の集合として使う。係数標識から評価標識へ移る操作が、ここで扱う有限体上のフーリエ変換である。

参考：複素数のフーリエ変換

物理や信号処理では、連続変数の関数を複素指数関数で分解するフーリエ変換がよく使われる。典型的には次のような積分で書く。

$$\widehat{f}(\xi) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i \xi t} dt$$

離散的な場合には、 $\zeta = e^{2\pi i/n}$ を使って

$$\widehat{a}_j = \sum_{i=0}^{n-1} a_i \zeta^{ij}$$

のような和で書ける。

今回使う有限体上のフーリエ変換は、この式の ζ を有限体の位数 n の 1 の根 ω に置き換えたものとして読む。角度や波ではなく、掛け算で一周する点列 $1, \omega, \dots, \omega^{n-1}$ を使う。

有限体上のフーリエ変換を定義する

係数列 $a = (a_0, a_1, \dots, a_{n-1})$ は、次数 n 未満の多項式

$$f(X) = \sum_{i=0}^{n-1} a_i X^i$$

を表す。位数 n の 1 の根 ω に対して、有限体上のフーリエ変換と逆変換を次で定義する。

$$\text{フーリエ変換:} \quad y_j = f(\omega^j) = \sum_{i=0}^{n-1} a_i \omega^{ij}, \quad j = 0, \dots, n-1.$$

$$\text{逆変換:} \quad a_i = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij}, \quad i = 0, \dots, n-1.$$

ここで $1/n$ は、有限体の中での n の逆元である。つまり、フーリエ変換は係数列 a から評価値列 y を作り、逆変換は評価値列 y から係数列 a を戻す。

逆変換で使う補題

n は $\mathbb{F}_p$ で逆元を持つとする。位数 n の 1 の根 ω について、次が成り立つ。

$$S_k = \sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n & (k \equiv 0 \pmod{n}), \\ 0 & (k \not\equiv 0 \pmod{n}). \end{cases}$$

$$\begin{aligned} k \not\equiv 0 \pmod{n}: \quad (1 - \omega^k) S_k &= \sum_{j=0}^{n-1} \omega^{kj} - \sum_{j=0}^{n-1} \omega^{k(j+1)} \\ &= 1 - \omega^{kn} = 0. \end{aligned}$$

$\omega^k \neq 1$ なので $S_k = 0$ である。 $k \equiv 0$ のときは各項が 1 なので $S_k = n$ である。

逆変換が元の係数を返すこと

$y_j = f(\omega^j) = \sum_{\ell=0}^{n-1} a_\ell \omega^{\ell j}$ とすると、

$$\begin{aligned}\frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij} &= \frac{1}{n} \sum_{j=0}^{n-1} \left(\sum_{\ell=0}^{n-1} a_\ell \omega^{\ell j} \right) \omega^{-ij} \\ &= \frac{1}{n} \sum_{\ell=0}^{n-1} a_\ell \sum_{j=0}^{n-1} \omega^{(\ell-i)j} \\ &= \frac{1}{n} \left(a_i n + \sum_{\ell \neq i} a_\ell \cdot 0 \right) = a_i.\end{aligned}$$

小例: $\mathbb{F}_7$ で 3 点に評価する

$\mathbb{F}_7$ では $\omega = 2$ が位数 3 の 1 の根になる。

$$2^0 = 1, \quad 2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 1$$

多項式

$$f(X) = 3 + X + 2X^2$$

を $H = \{1, 2, 4\}$ 上で評価すると、

$$f(1) = 6, \quad f(2) = 6, \quad f(4) = 4 \pmod{7}$$

何を変換したか

係数列 $(3, 1, 2)$ を、評価値列 $(6, 6, 4)$ へ変換した。

FFT: フーリエ変換を速く計算する

フーリエ変換の式

$$y_j = \sum_{i=0}^{n-1} a_i \omega^{ij} \quad (j = 0, \dots, n-1)$$

をそのまま計算すると、1点あたり n 回の掛け算を n 点分繰り返すので、掛け算はおよそ n^2 回かかる。

方法	掛け算の回数	$n = 2^{20}$ のとき
定義どおりの計算	n^2	約 10^{12} 回
FFT	$\frac{n}{2} \log_2 n$	約 10^7 回

証明生成では n が 2^{20} を超える変換を何度も行うため、この差が実用性を左右する。FFT は、評価点集合 $H = \{1, \omega, \dots, \omega^{n-1}\}$ の規則性を利用して、同じ y を高速に求めるアルゴリズム。以降、2つの観察を積み上げてアルゴリズムを組み立てる。

観察 1: 評価点は $\pm$ の対をなす

n を偶数とする。 $\omega^{n/2}$ は 2 乗すると $\omega^n = 1$ になり、位数 n の仮定から 1 とは異なるので

$$\omega^{n/2} = -1, \quad \omega^{j+n/2} = -\omega^j.$$

つまり評価点集合 H は $n/2$ 組の対 $\pm\omega^j$ ($j = 0, \dots, n/2 - 1$) に分かれる。

さらに、対になった 2 点は 2 乗すると同じ点に移る。

$$(\pm\omega^j)^2 = \omega^{2j}$$

ω^2 は位数 $n/2$ の 1 の根なので、2 乗後の点集合は半分のサイズの評価点集合 $H' = \{1, \omega^2, \omega^4, \dots\}$ になる。

例: $\mathbb{F}_5$ で $\omega = 2$ (位数 4)

$H = \{1, 2, 4, 3\}$ 。 $4 = -1$ 、 $3 = -2$ なので対は $(1, 4)$ と $(2, 3)$ 。 2 乗すると $\{1, 4\}$ の 2 点に半減する。

観察 2: 多項式は偶数次と奇数次に分けられる

係数を偶数次と奇数次に振り分けると、どんな f も次の形に書ける。

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$$

例: $f(X) = 1 + 2X + 3X^2 + 4X^3$ なら

$$f_{\text{even}}(Y) = 1 + 3Y, \quad f_{\text{odd}}(Y) = 2 + 4Y, \quad f(X) = (1 + 3X^2) + X(2 + 4X^2).$$

この形に対する 2 点 $X = \pm\omega^j$ を代入すると、 $X^2 = \omega^{2j}$ が共通なので

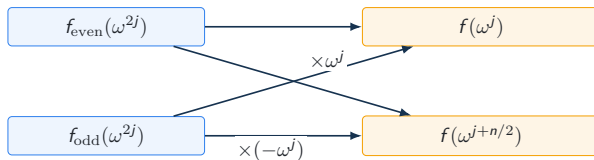
$$f(\omega^j) = f_{\text{even}}(\omega^{2j}) + \omega^j f_{\text{odd}}(\omega^{2j}), \quad f(-\omega^j) = f_{\text{even}}(\omega^{2j}) - \omega^j f_{\text{odd}}(\omega^{2j}).$$

半分サイズの多項式 $f_{\text{even}}, f_{\text{odd}}$ の値を 1 回ずつ求めれば、2 点分の値が同時に得られる。

蝶形: 2つの観察を組み合わせる

$E_j = f_{\text{even}}(\omega^{2j})$ 、 $O_j = f_{\text{odd}}(\omega^{2j})$ と置くと、 $j = 0, \dots, n/2 - 1$ について

$$y_j = E_j + \omega^j O_j, \quad y_{j+n/2} = E_j - \omega^j O_j.$$



この組み合わせ方を蝶形 (butterfly) と呼ぶ。掛け算は $\omega^j O_j$ の1回だけ。残る仕事は E_j, O_j を求めること、つまり $f_{\text{even}}, f_{\text{odd}}$ を H' 上で評価することで、これは半分のサイズ $n/2$ の同じ問題が2つ。同じ手順を再帰的に繰り返せばよい。

再帰の全体像と計算量

サイズ n の変換は、蝶形 $n/2$ 個とサイズ $n/2$ の変換 2 つに置き換わる。サイズ 1 まで割り切ったら、定数多項式の値は係数そのものなので計算は終わる。

段	部分問題	蝶形の掛け算
1 段目	サイズ n が 1 つ	$n/2$ 回
2 段目	サイズ $n/2$ が 2 つ	$n/2$ 回
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$ 段目	サイズ 2 が $n/2$ 個	$n/2$ 回

どの段でも蝶形の掛け算は合計 $n/2$ 回で、段数は $\log_2 n$ 。掛け算の総数は

$$\frac{n}{2} \log_2 n.$$

半分に割り続けるため、radix-2 FFT では $n = 2^m$ となる評価点集合を選ぶ。

手で追う例 (1/2): $\mathbb{F}_5$ 、 $n = 4$ で準備する

半分に割り続けるにはサイズが 2^m の例が要る。 $\mathbb{F}_5$ では $\omega = 2$ が位数 4 の 1 の根になる。

$$2^0 = 1, \quad 2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 3, \quad 2^4 = 1$$

係数列 $a = (1, 2, 3, 4)$ 、つまり

$$f(X) = 1 + 2X + 3X^2 + 4X^3$$

を $H = \{1, 2, 4, 3\}$ 上で評価する。

Step 1: 偶数次と奇数次に分ける

$$f_{\text{even}}(Y) = 1 + 3Y, \quad f_{\text{odd}}(Y) = 2 + 4Y$$

この 2 つを、半分の評価点集合 $H' = \{1, \omega^2\} = \{1, 4\}$ 上で評価すればよい。

手で追う例 (2/2): 蝶形で組み上げる

Step 2: サイズ 2 の評価 ($\omega^2 = 4 = -1$ を代入するだけ)

$$\begin{aligned} E_0 &= f_{\text{even}}(1) = 1 + 3 = 4, & E_1 &= f_{\text{even}}(4) = 1 + 3 \cdot 4 = 13 = 3 \\ O_0 &= f_{\text{odd}}(1) = 2 + 4 = 6 = 1, & O_1 &= f_{\text{odd}}(4) = 2 + 4 \cdot 4 = 18 = 3 \end{aligned}$$

Step 3: 蝶形でサイズ 4 に合成 ($\omega = 2$)

$$\begin{aligned} y_0 &= E_0 + \omega^0 O_0 = 4 + 1 = 5 = 0, & y_2 &= E_0 - \omega^0 O_0 = 4 - 1 = 3 \\ y_1 &= E_1 + \omega^1 O_1 = 3 + 2 \cdot 3 = 9 = 4, & y_3 &= E_1 - \omega^1 O_1 = 3 - 2 \cdot 3 = -3 = 2 \end{aligned}$$

結果は $y = (0, 4, 3, 2)$ 。検算として定義どおりに $f(2) = 1 + 4 + 12 + 32 = 49 = 4$ を計算すると y_1 と一致する。

IFFT と押さえどころ

逆変換

$$a_i = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij}$$

は、 ω を ω^{-1} に置き換えた同じ形の和である。 ω^{-1} も位数 n の 1 の根なので、同じ FFT を実行して最後に各成分へ $1/n$ を掛ければよい。これを IFFT と呼ぶ。

最低限覚えること

FFT/IFFT は、係数列と評価値列の行き来を $\frac{n}{2} \log_2 n$ 回の掛け算で行う。鍵は「評価点が $\pm$ の対をなす」ことと「偶数次・奇数次への分割」で、サイズ n の変換が半分のサイズの変換 2 つに落ちる。

点の足し算で群を作る

数の準備では、 $\mathbb{F}_p^*$ が掛け算について群になることを見た。楕円曲線は、曲線上の点の「足し算」で群を作る、もう一つの方法である。

点の足し算

$$P + Q = R$$

点と点を足すと、また曲線上の点になる。

スカラー倍

$$xP = \underbrace{P + \cdots + P}_{x \text{ 個}}$$

P を x 回足す操作として読む。

この群が暗号の土台になるのは、 xP から x を逆算するのが難しいため。このセクションでは、足し算の作り方から順に見て、この一方向性へたどり着く。

楕円曲線を式で見る

暗号でよく使う楕円曲線は、有限体 $\mathbb{F}_p$ 上の点集合として定義される。

$$y^2 = x^3 + ax + b$$

滑らかさの条件

$$4a^3 + 27b^2 \neq 0$$

この条件により、特異点のない曲線として扱える。

$$E(k) = \{(x, y) \in k^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

$\mathcal{O}$ は無限遠点であり、足し算の単位元として働く。

点の足し算の式: 3つ目の交点

$P = (x_1, y_1)$ 、 $Q = (x_2, y_2)$ とする。

定義

P, Q を通る直線が曲線と 3 つ目に交わる点を $R' = (x_3, y')$ とする。 R' の x 軸反転 $R = (x_3, -y')$ を

$$P + Q := R$$

と定める。

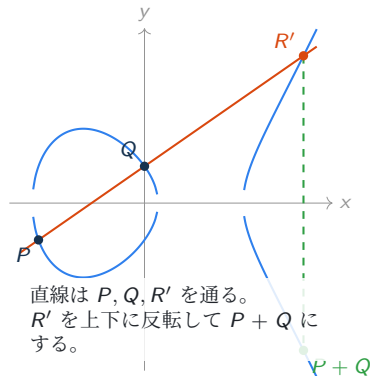
ここから x_3, y' を式で求める。 $P \neq Q$ かつ $x_1 \neq x_2$ の通常ケースでは、

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

とおく。直線 $y = \lambda(x - x_1) + y_1$ を曲線 $y^2 = x^3 + ax + b$ に代入すると、

$$\{\lambda(x - x_1) + y_1\}^2 = x^3 + ax + b$$

という 3 次方程式になる。解は交点の x 座標 x_1, x_2, x_3 である。



図は実数上の直感。暗号では同じ式を有限体で $\text{mod } p$ 計算する。

点の足し算の式: 反転して和を作る

代入して得た 3 次方程式を整理すると、 x^2 の係数は $-\lambda^2$ になる。根と係数の関係から

$$x_1 + x_2 + x_3 = \lambda^2$$

なので、

$$x_3 = \lambda^2 - x_1 - x_2$$

である。

定義で置いた 3 つ目の交点 $R' = (x_3, y')$ は直線上にあるので、

$$y' = \lambda(x_3 - x_1) + y_1$$

である。 $P + Q = (x_3, y_3)$ は R' の x 軸反転なので、したがって

$$y_3 = -y' = -\{\lambda(x_3 - x_1) + y_1\} = \lambda(x_1 - x_3) - y_1.$$

$P = Q$ の点倍算では、直線の代わりに接線を使い、

$$\lambda = \frac{3x_1^2 + a}{2y_1}$$

とする。ただし $2y_1 \neq 0$ の場合である。 $y_1 = 0$ なら接線は垂直になり、 $2P = \mathcal{O}$ と扱う。割り算は有限体の逆元で行う。

手で追う例: $\mathbb{F}_{11}$ 上で $2P$ を計算する

曲線 $y^2 = x^3 + x + 6$ ($\mathbb{F}_{11}$ 上、 $a = 1$) で、点 $P = (2, 7)$ の 2 倍を接線の式で計算する。

Step 1: 接線の傾き λ

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{3 \cdot 4 + 1}{14} = 13 \cdot 14^{-1} \equiv 2 \cdot 3^{-1} \equiv 2 \cdot 4 = 8 \pmod{11}$$

ここで $3^{-1} = 4$ は $3 \cdot 4 = 12 \equiv 1 \pmod{11}$ から。

Step 2: 座標を求める

$$x_3 = \lambda^2 - 2x_1 = 64 - 4 = 60 \equiv 5, \quad y_3 = \lambda(x_1 - x_3) - y_1 = 8 \cdot (2 - 5) - 7 = -31 \equiv 2$$

よって $2P = (5, 2)$ 。検算: $2^2 = 4$ 、 $5^3 + 5 + 6 = 136 \equiv 4 \pmod{11}$ で、たしかに曲線上の点。

スカラー倍は倍々で速くなる

xP は P を x 回足す操作。定義どおりに足すと $x - 1$ 回の演算がかかる。実装では x の二進展開を使う。

例: $13P(13 = (1101)_2 = 8 + 4 + 1)$

$$P \xrightarrow{\text{doubling}} 2P \xrightarrow{\text{doubling}} 4P \xrightarrow{\text{doubling}} 8P, \quad 13P = 8P + 4P + P$$

doubling 3 回と足し算 2 回、合計 5 回の演算で届く。定義どおりなら 12 回かかる。

一般に演算回数は $\log_2 x$ に比例する。 x が 256bit 程度の巨大な数でも、数百回の演算で xP を計算できる。

楕円曲線の離散対数問題

楕円曲線暗号では、曲線上の点を足し算できる。

- 点 P と Q を足して $P + Q$ を得る。
- 点 P を x 回足して xP を得る。

Given P and $Q = xP$, find x

この問題を楕円曲線上の離散対数問題と呼ぶ。

暗号では、基点 P が生成する大きな素數位数 q の部分群を使い、 $x \in \mathbb{Z}_q$ として扱う。

一方向性がある

$x \mapsto Q = xP$ は速い

$P, Q = xP \nrightarrow x$ は難しい

これは hash と同じ見方である。hash でも $m \mapsto \text{Hash}(m)$ は速いが、 $\text{Hash}(m)$ から m を戻すのは難しい。

楕円曲線では、入力をスカラー x 、出力を点 xP としてこの構造を使う。

離散対数問題の仲間たち

離散対数問題の周りには、この後の安全性の議論で語彙になる仲間の問題がある。Alice と Bob がそれぞれ aP 、 bP を公開して共通の abP を作る鍵共有を思い浮かべると覚えやすい。乗法記法の g^a と加法記法の aP は、同じ操作の書き方違いである。

問題	乗法記法	加法記法
離散対数問題	g, g^a から a を求める	P, aP から a を求める
計算 Diffie–Hellman 問題	g, g^a, g^b から g^{ab} を求める	P, aP, bP から abP を求める
判定 Diffie–Hellman 問題	g, g^a, g^b, T から $T = g^{ab}$ かランダムな g^c かを判定する	P, aP, bP, T から $T = abP$ かランダムな cP かを判定する

記法は違っても、秘密スカラーを公開できる群要素へ写すという構造は同じである。

対話で証明する

導入のソムリエの例は、そのまま対話式ゼロ知識証明の形をしている。verifier がチャレンジ (入れ替えるか) を選び、prover が応答し、繰り返して納得が積み上がる。

このセクションの道筋

- ① 数式で書ける最小の対話式証明、Schnorr プロトコルを 1 ステップずつ追う。
- ② 導入で定式化した知識健全性と zero-knowledge を、Schnorr の上で具体的に確かめる。
- ③ Fiat-Shamir 変換で対話を消し、proof1 通の証明にする。zkSNARK の「N」がここで手に入る。

材料は前のセクションまでに揃っている。巡回群と生成元、そして離散対数の一方方向性である。

Schnorr: 問題設定

公開情報として、巡回群の生成元 g と $y = g^x$ がある。prover は秘密の x を知っていることを示したい。導入の言葉で書くと、次の statement と witness の組である。

statement: $\exists x : g^x = y$, witness: x

目的

x を明かさずに、 y が g^x という形で作られていることを納得させる。

ここでは乗法記法で書く。楕円曲線では g^x を xP 、 g^r を rP と読めばよい。群の位数を q とし、指数 r, x, b は $\text{mod } q$ で計算する。

Schnorr: 対話式プロトコル

- ① prover は乱数 r を選び、 $a := g^r$ を verifier へ送る。
- ② verifier はチャレンジ b をランダムに選んで送る。
- ③ prover は $c := r + bx$ を返す。
- ④ verifier は

$$g^c = ay^b$$

を確認する。

完全性

本当に $y = g^x$ なら、

$$g^c = g^{r+bx} = g^r (g^x)^b = ay^b$$

となる。

手で追う例: 小さな数で Schnorr を回す

$\mathbb{F}_{23}^*$ の中で、位数 $q = 11$ の元 $g = 2$ を使う。秘密 $x = 7$ 、公開値 $y = 2^7 = 128 \equiv 13 \pmod{23}$ 。

1 回の対話

- ① prover: 乱数 $r = 3$ を選び、 $a = 2^3 = 8$ を送る。
- ② verifier: チャレンジ $b = 5$ を送る。
- ③ prover: $c = r + bx = 3 + 5 \cdot 7 = 38 \equiv 5 \pmod{11}$ を返す。
- ④ verifier: 左辺は $g^c = 2^5 = 32 \equiv 9 \pmod{23}$ 。右辺は $13^2 \equiv 8$, $13^4 \equiv 8^2 \equiv 18$, $13^5 \equiv 18 \cdot 13 \equiv 4$ より $ay^b = 8 \cdot 4 = 32 \equiv 9$ 。両辺が一致するので受理。

指数の計算は $\text{mod } q = 11$ 、群の計算は $\text{mod } p = 23$ で行っている点に注意。

Schnorr: 知識健全性

同じ a に対して、異なるチャレンジ $b \neq b'$ の両方に答えられる prover からは、秘密 x を取り出せる。
2つの応答 c, c' がどちらも検証に通るなら、

$$c = r + bx, \quad c' = r + b'x \pmod{q}$$

の形をしている。差を取ると r が消えて

$$c - c' = (b - b')x$$

となり、 $b \neq b'$ なので $b - b'$ の逆元を掛けて

$$x = (c - c')(b - b')^{-1} \pmod{q}$$

が手に入る。

巻き戻しの見方

prover を同じ a を送った時点まで戻し、別のチャレンジを投げる。両方に答えられるなら、prover は実質的に x を知っている。定式化 2 で見た抽出器 Ext を、Schnorr の上で具体的に作ったことになる。

Schnorr: honest-verifier zero-knowledge

正直な verifier がランダムな b を送る場合、verifier が見る会話 (a, b, c) は秘密 x を使わずに作れる形をしている。

simulator の作り方

b, c を先にランダムに選び、

$$a := g^c y^{-b}$$

と置く。

本物の実行では $c = r + bx$ なので、

$$a = g^r = g^{c-bx} = g^c (g^x)^{-b} = g^c y^{-b}$$

と同じ形になる。

すると検証式

$$g^c = ay^b$$

は必ず成り立つ。公開情報だけで本物と見分けにくい会話を作れることが、honest-verifier zero-knowledge の核になる。定式化 3 で見た simulator が、Schnorr では 1 行で書けたことになる。

Fiat-Shamir 変換: チャレンジを hash で作る

対話式プロトコルでは、verifier が後からランダムなチャレンジ b を送る。Fiat-Shamir 変換では、このチャレンジをハッシュで作る。

$$b := H(\text{ctx}, g, y, a)$$

ctx に入れるもの

プロトコル名、曲線や群の種類、証明したい statement などの文脈情報を入れる。

transcript は公開された会話ログである。transcript hash により、verifier が送るはずのチャレンジを公開情報から決定論的に再計算できる。

非対話になった Schnorr

Fiat-Shamir 後の prover:

- ① 乱数 r を選び、 $a := g^r$ を計算する。
- ② $b := H(\text{ctx}, g, y, a)$ を計算する。
- ③ $c := r + bx \pmod{q}$ を計算する。
- ④ proof として $\pi = (a, c)$ を送る。

verifier は a から同じ b を再計算し、

$$g^c = ay^b$$

を確認する。

実装で重要なこと

domain separation、値の一意なエンコード、十分に大きなチャレンジ空間を用意する。

なぜペアリングが要るのか

楕円曲線の点 aP は、スカラー a を隠したまま公開できるのだった。一方、この後の commitment では「隠れたスカラーどうしの掛け算の関係」を確かめたい場面が出てくる。

困りごと

aP と bP が手元にあっても、点の足し算だけでは abP を作れない。足し算で作れるのは $aP + bP = (a + b)P$ まで。掛け算の情報には届かない。

ペアリングは、点のペアから「スカラーの積」を検査できる値を作る演算である。積そのものを取り出す道具ではない点が肝で、離散対数の一方向性は保たれる。

ペアリングの定義

ペアリングは、二つの群の要素を別の群 G_T へ写す演算。 G_1, G_2, G_T の位数はどれも素数 q とする。

$$e : G_1 \times G_2 \rightarrow G_T$$

双線形性: スカラーを指数へ移せる

$$e(aP, Q) = e(P, Q)^a = e(P, aQ)$$

どちらの引数に付いたスカラーも G_T の指数として外へ出せる。両方まとめると $e(aP, bQ) = e(P, Q)^{ab}$ 。

- 非退化性: 生成元 P, Q に対して $e(P, Q) \neq 1$ 。
- 効率的計算可能性: P, Q から $e(P, Q)$ を効率よく計算できる。
- 実用では、 G_1, G_2, G_T を別々の群として扱う非対称ペアリングがよく使われる。

使い方: 積の関係を等式で確かめる

aP, bQ, cP を受け取った verifier が、「 $c \equiv ab \pmod{q}$ か」を確かめたいとする。双線形性で両辺を $e(P, Q)$ の指数に直すと

$$e(aP, bQ) = e(P, Q)^{ab}, \quad e(cP, Q) = e(P, Q)^c$$

なので、

$$e(aP, bQ) = e(cP, Q) \iff ab \equiv c \pmod{q}$$

として検査できる。スカラー a, b, c そのものは最後まで登場しない。

次のセクションで見る KZG commitment の検証式 $e(C - yP, Q) = e(\pi, \tau Q - zQ)$ は、まさにこの「指数の等式をペアリングで確かめる」形をしている。

Commitment は zkSNARK で何をしているか

証明システムでは、大きな対象が何度も出る。

- 制約の列
- 実行トレース
- 多項式
- wire 値や selector

役割

commitment は、大きな対象を短い値で代表させ、必要な点だけを open するために使う。

このセクションでは、hash、Merkle tree、Pedersen、KZG の順に構成を積み上げる。



開示する部分だけを
検証対象にする

Commitment scheme

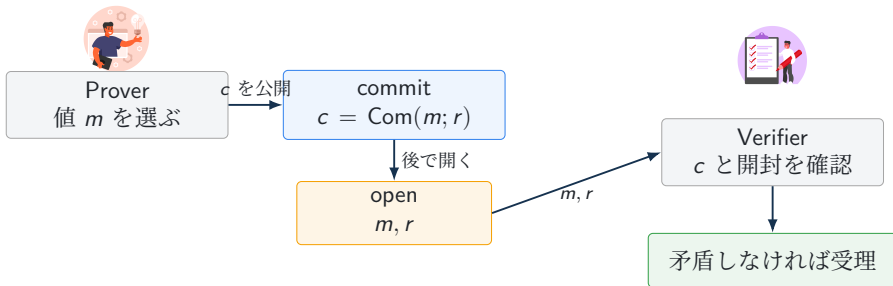
commitment は「封をした封筒」である。値 m を短い値 c に固定して先に渡し、後で m と乱数 r を見せて開ける。

$$c \leftarrow \text{Com}(m; r), \quad \text{Verify}(c, m, r) \in \{0, 1\}$$

Binding: 封をした後で中身を差し替えられない。commit 後に、別の値として open できない。

Hiding: 開けるまで中身が見えない。open するまで、 c から m が漏れない。

基本形では全体を開くが、Merkle tree や KZG では位置や点の値だけを補助情報つきで open する。



Commitment の定式化

封筒の直感を、3つのアルゴリズムと2つの性質の式に写す。公開パラメータを使う構成では、次のように書ける。

$$pp \leftarrow \text{Setup}(1^\lambda), \quad (c, d) \leftarrow \text{Commit}(pp, m), \quad b \leftarrow \text{Verify}(pp, c, m, d)$$

記号	意味
pp	公開パラメータ
c	commitment
d	decommitment / opening information。乱数、path、proof などの確認用補助情報
b	検証結果。受理なら 1、拒否なら 0

Binding

効率的な攻撃者が、同じ commitment を二つの異なる message として open することは困難である。このような組を見つける確率は negligible である。

$$\text{Verify}(pp, c, m, d) = 1,$$

$$\text{Verify}(pp, c, m', d') = 1, \quad m \neq m'$$

Hiding

c だけを見ても、どちらの message から作られたかを当てにくい。

$$\left| \Pr[b' = b] - \frac{1}{2} \right| \text{ が小さい}$$

Hash commitment の定義

Hash commitment は、message m を hash 値 c に固定する commitment である。

$$c = \text{Hash}(m)$$

prover は c を先に公開する。後で open するとき
は m を渡し、verifier は

$$\text{Hash}(m) = c$$

を確認する。

- commitment は hash 値 1 個で短い。
- open まで m は prover の手元に残る。
- 安全性は、次に見る hash 関数の性質から来る。

Hash commitment の安全性

Hash commitment では

$$c = \text{Hash}(m)$$

として値を固定する。

性質	意味
一方向性	$h = \text{Hash}(m)$ から元の m を見つけにくい。
第二原像困難性	与えられた m と同じ hash になる別の $m' \neq m$ を見つけにくい。
衝突困難性	異なる m, m' で $\text{Hash}(m) = \text{Hash}(m')$ となる組を見つけにくい。

binding には主に第二原像困難性や衝突困難性が効く。一方、hiding はこの表からは出てこない。次のスライドで対策を見る。

Hash commitment で値を隠すには

$c = \text{Hash}(m)$ は、候補が少ないと隠せない。たとえば賛成・反対の投票 $m \in \{0, 1\}$ なら、verifier は両方を hash して c と見比べるだけで中身を当てられる。

乱数を混ぜる

秘匿性が必要な場合は、十分長い乱数 r を混ぜる。

$$c = \text{Hash}(\text{encode}(\text{domain}, r, m))$$

- domain: 用途を区別する文字列として入れる。
- encode: 長さや区切りが曖昧にならない一意な符号化を使う。

Merkle tree

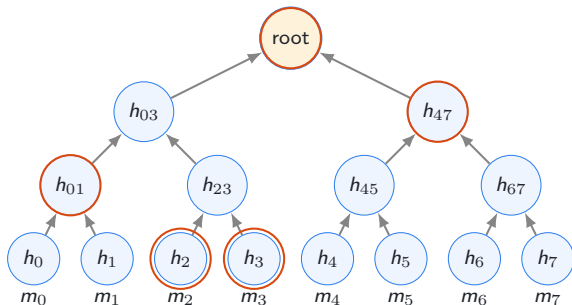
大きな配列 $(m_0, m_1, \dots, m_{n-1})$ 全体を固定したいとき、配列全体を毎回 open すると検証の負担が大きい。Merkle tree は、葉から親へ hash を重ねて、配列全体を 1 つの root にまとめる。

root を commitment として公開する。

$$h_i = \text{Hash}(m_i), \quad \text{root} = \text{Hash}(\dots)$$

1 要素だけを open するときは、root まで再計算するために必要な隣接 hash だけを渡す。

Merkle tree では、配列の一部だけを $O(\log n)$ の proof で open できる。



手で追う例: Merkle proof の検証

8 個の leaf の木で、 $root$ と index 2 が公開されているとする。prover が渡すのは、開く値 m_2 と、道中の隣接 hash 3 個 (path)。

$$m_2, \quad (h_3, \text{right}), \quad (h_{01}, \text{left}), \quad (h_{47}, \text{right})$$

verifier の再計算: 葉から root へ上る

$$h_2 = \text{Hash}(m_2), \quad h_{23} = \text{Hash}(h_2 \parallel h_3), \quad h_{03} = \text{Hash}(h_{01} \parallel h_{23}), \quad root' = \text{Hash}(h_{03} \parallel h_{47})$$

最後に $root' = root$ を確認する。一致すれば、 m_2 が index 2 に固定されていたと納得できる。

渡す hash は木の高さぶんの $\log_2 8 = 3$ 個。leaf が 100 万個でも 20 個で済む。

実装上の注意

左右の順序、leaf のエンコード、domain separation を曖昧にしない。

Pedersen commitment

楕円曲線上の2つの基点 G, H を使い、値 m と乱数 r をスカラー倍で混ぜる。

$$\text{Com}(m; r) = mG + rH$$

Hiding

乱数 r によって、同じ m でも異なる点になる。

Binding

G と H の離散対数関係が知られていないことが重要になる。

加法準同型性: 足し算がそのまま通る

$$\text{Com}(a; r_a) + \text{Com}(b; r_b) = (a + b)G + (r_a + r_b)H = \text{Com}(a + b; r_a + r_b)$$

commitment を開かずに、中身の和の commitment を作れる。

Pedersen の binding は離散対数に立つ

同じ commitment を二通りに開けたと仮定する。

$$mG + rH = m'G + r'H, \quad (m, r) \neq (m', r')$$

移項すると

$$(m - m')G = (r' - r)H$$

となり、 $H = \alpha G$ と書いたときの $\alpha = (m - m')(r' - r)^{-1}$ が計算できてしまう。つまり二重 open は、 H の離散対数を解いたのと同じこと。離散対数問題が難しく、誰も α を知らない限り、binding は破れない。

もう一つの注意

値は群の位数 q を法として扱われる。整数としての範囲を保証したい場合は range proof や追加制約が必要になる。

SRS

SRS は Structured Reference String の略で、証明者と検証者が共通に使う公開パラメータである。KZG では、setup で秘密の τ を選び、 τ から作った群要素だけを公開する。

$$\text{SRS} = (P, \tau P, \tau^2 P, \dots, \tau^d P; Q, \tau Q)$$

commitment と proof 生成には $\tau^i P$ のような G_1 側の群要素を使う。prover も verifier もこれを使う。ただし、群要素から τ そのものを取り出せないことを使っている。

pairing 検証では、検証用に $Q, \tau Q$ も公開される。 τ そのものは知られてはいけない。 τ が漏れると、主に binding が壊れ、不正な opening proof を作れる可能性がある。

trusted setup は、 τ を安全に生成し、最後に破棄したと信じる必要がある設定である。この秘密は toxic waste と呼ばれることがある。

KZG commitment

KZG は、多項式 $f(X)$ を短い commitment C に固定し、後で 1 点での値 $f(z) = y$ を短い proof で示す多項式 commitment である。

SRS として次の群要素が公開されているとする。

$$P, \tau P, \tau^2 P, \dots, \tau^d P; \quad Q, \tau Q$$

多項式を次のように書く。

$$f(X) = a_0 + a_1 X + \dots + a_d X^d$$

この多項式の commitment は

$$C = a_0 P + a_1 (\tau P) + \dots + a_d (\tau^d P) = f(\tau) P$$

証明者も検証者も τ 自体は知らない。公開された群要素を線形結合することで、 $f(\tau)P$ に相当する点を作る。

KZG opening

$f(z) = y$ を示したい。因数定理より、 $f(X) - y$ は $X - z$ で割り切れる。

$$q(X) = \frac{f(X) - y}{X - z}$$

KZG proof は

$$\pi = q(\tau)P$$

検証式:

$$e(C - yP, Q) = e(\pi, \tau Q - zQ)$$

正しく作った proof なら、completeness は次の式変形で確認できる。

$$\begin{aligned} e(C - yP, Q) &= e((f(\tau) - y)P, Q) \\ &= e(q(\tau)(\tau - z)P, Q) \\ &= e(q(\tau)P, (\tau - z)Q) \\ &= e(\pi, \tau Q - zQ). \end{aligned}$$

この式変形は、ペアリングの節の「スカラーをどちらの引数からも指数へ移せる」規則だけでできている。 C も π も群要素 1 個で、多項式の次数によらず定数サイズ。

手で追う例: $f(X) = X^2 + 1$ で $f(2) = 5$ を開く

Step 1: commitment

$$C = f(\tau)P = \tau^2 P + P$$

多項式の準備で補間した $f(X) = X^2 + 1$ を固定する。SRS にある $\tau^2 P$ と P を足すだけで、 τ を知らずに計算できる。

Step 2: 商多項式と proof

$$q(X) = \frac{f(X) - 5}{X - 2} = \frac{X^2 - 4}{X - 2} = X + 2, \quad \pi = q(\tau)P = \tau P + 2P$$

割り切れるのは、 $f(2) = 5$ が本当に成り立つときだけ。

Step 3: 検証

$$e(C - 5P, Q) = e(\pi, \tau Q - 2Q)$$

左辺の指数は $f(\tau) - 5 = \tau^2 - 4$ 、右辺の指数は $(\tau + 2)(\tau - 2) = \tau^2 - 4$ で一致する。

参考: KZG の hiding 版

基本版の $C = f(\tau)P$ は乱数を含まず、hiding がない。値を隠したいときは、独立な基点 H と乱数多項式 $r(X)$ を混ぜる。

$$C = f(\tau)P + r(\tau)H$$

SRS には $H, \tau H, \dots$ 側の powers も必要になる。

$f(z) = y$, $r(z) = \rho$ を開くときは、商を 2 本作って束ねる。

$$q_f(X) = \frac{f(X) - y}{X - z}, \quad q_r(X) = \frac{r(X) - \rho}{X - z}, \quad \pi = q_f(\tau)P + q_r(\tau)H$$

検証式は基本版と同じ形になる。

$$e(C - yP - \rho H, Q) = e(\pi, \tau Q - zQ)$$

Pedersen で乱数 r を混ぜたのと同じ発想である。

参考: KZG の batch 版

複数の点 $z_1, \dots, z_m$ での値 $f(z_i) = y_i$ を、proof1 個でまとめて示せる。補間多項式 $Y(X)$ と積多項式 $D(X)$ を置く。

$$Y(z_i) = y_i, \quad D(X) = \prod_{i=1}^m (X - z_i)$$

すべての z_i で $f(z_i) = y_i$ が成り立つことは、 $f(X) - Y(X)$ が $D(X)$ で割り切れることと言い換えられる。

$$q(X) = \frac{f(X) - Y(X)}{D(X)}, \quad \pi = q(\tau)P$$

検証は 1 つの pairing 等式で行う。

$$e(C - Y(\tau)P, Q) = e(\pi, D(\tau)Q)$$

この式をそのまま使うには、 $D(\tau)Q$ を作るための G_2 側 powers も必要になる。

Commitment scheme の比較

Scheme	対象	主な仮定	証明サイズ	強み	注意点
Hash	1 個の message	第二原像困難性、衝突困難性	open 対象に依存	単純	小さい message は乱数が必要になる
Merkle	配列・木	第二原像困難性、衝突困難性	$O(\log n)$	部分 open が得意	順序、salt、domain separation
Pedersen	値・ベクトル	離散対数問題、基点間の未知関係	開く値に依存	hiding、加法準同型	乱数再利用、範囲制約
KZG	多項式	ペアリング、SDH 系仮定、SRS	π は $O(1)$	短い proof、高速検証	opening data と検証計算は点数に依存

最低限覚えること

Merkle はデータ構造、Pedersen は準同型 commitment、KZG は多項式 commitment として覚える。

Arithmetization の目的

arithmetization は、

計算が正しく実行された

という主張を、有限体上の制約として表す工程である。
zk アプリケーション開発では、「自分が書いたコードがどんな制約になるか」を意識することが重要になる。

このセクションでは、制約の書き方から始め、R1CS・AIR・Plonkish・CCS という 4 つの流儀を順に見る。



Prover が扱う計算を
制約へ落とす

制約は「0になる式」として書く

制約は、候補の値を代入したときに成り立つべき条件である。verifier は計算をもう一度実行する代わりに、値どうしの整合性を等式で確認する。

計算として読む

$$a + b = c$$

c は $a + b$ の結果である。

制約として読む

$$a + b - c = 0$$

与えられた a, b, c が整合しているかを確認する。

確認しやすいように、成り立つべき関係は「左辺が0になる式」へそろえる。

$$ab - d = 0, \quad b(b - 1) = 0 \text{ (つまり } b \in \{0, 1\})$$

AND なら $ab - c = 0$ と書き、 a, b, c が boolean であることも別の制約として入れる。

witness と中間値を制約へ分解する

次の計算を証明したい。

$$x^3 + x + 5 = y$$

verifier にも見える値は y であり、prover は秘密入力 x と中間値 t_1, t_2 を witness として持つ。

public input: $y = 15$ witness variables: x, t_1, t_2

計算ステップへ分解する

$$t_1 \leftarrow x \cdot x$$

$$t_2 \leftarrow t_1 \cdot x$$

$$y \leftarrow t_2 + x + 5$$

たとえば $x = 2$ なら、

$$t_1 = 4, \quad t_2 = 8, \quad y = 15$$

制約として読む

$$t_1 - x^2 = 0$$

$$t_2 - t_1 x = 0$$

$$y - t_2 - x - 5 = 0$$

これらがすべて 0 なら、提出された x, t_1, t_2 は公開値 y と整合している。

線形結合とは何か

R1CS では「線形結合」という言葉が何度も出る。これは変数を係数付きで足した式である。

$$3z_1 + 5z_2 - z_4 + 7$$

定数項を扱いやすくするため、変数ベクトルの先頭に 1 を入れることが多い。

$$z = (1, y, x, t_1, t_2)$$

例

$t_2 + x + 5 - y$ は、 $z = (1, y, x, t_1, t_2)$ に対する線形結合として書ける。

$$5 \cdot 1 - 1 \cdot y + 1 \cdot x + 0 \cdot t_1 + 1 \cdot t_2$$

R1CS: 掛け算 1 個の制約へ分解する

R1CS の 1 本の制約は、

$$\text{線形結合} \times \text{線形結合} = \text{線形結合}$$

の形で書く。
変数ベクトル

$$z = (1, \text{public inputs, witness variables})$$

を用意し、

$$\langle A_i, z \rangle \cdot \langle B_i, z \rangle = \langle C_i, z \rangle$$

を全ての i で満たすかを見る。

感覚

R1CS は、算術回路を「掛け算 1 個を含む等式」の集まりへ分解する見方である。

R1CS で小例を行列として書く

$x^3 + x + 5 = y$ では $z = (1, y, x, t_1, t_2)^\top$ とおく。3本の制約を、3行の行列でまとめて書ける。

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 5 & -1 & 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

R1CS は次の形で確認する。

$$(Az) \circ (Bz) = Cz$$

実際に掛けると、

$$\begin{pmatrix} x \\ t_1 \\ t_2 + x + 5 - y \end{pmatrix} \circ \begin{pmatrix} x \\ x \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 \\ t_2 \\ 0 \end{pmatrix}.$$

これは $x \cdot x = t_1$ 、 $t_1 \cdot x = t_2$ 、 $t_2 + x + 5 - y = 0$ を表す。

同じ計算を違う形に並べる

ここから出る方式は、別々の数学をしているように見えるが、目的は同じ。

方式	何として並べるか	見方
R1CS	掛け算 1 個の制約リスト	回路を細かい等式へ分解する
AIR	時間方向の trace table	実行ログの各行が正しいか見る
Plonkish	wire 列、gate、selector の表	表計算のように行ごとのルールを見る

共通点

どの方式でも、計算の正しさを有限体上の制約として扱う。

AIR: 実行トレースと遷移制約

AIR は、計算を「横に変数、縦に時間」の表 (実行トレース) として見る。Fibonacci の例:

step	a	b
0	0	1
1	1	1
2	1	2
3	2	3

制約は 2 種類

- 遷移制約: 隣り合う行の関係。 $a_{i+1} = b_i$ 、 $b_{i+1} = a_i + b_i$ 。たとえば row 1 $\rightarrow$ 2 は $a_2 = b_1 = 1$ 、 $b_2 = a_1 + b_1 = 2$ で成立している。
- 境界制約: 始点と終点の値。 $a_0 = 0$, $b_0 = 1$ と、最終行が公開出力と一致すること。

R1CS は回路を見る。AIR は実行ログの全行が規則に従うかを見る。

Plonkish: 表として制約を書く

Plonkish arithmetization では、計算を表として配置する。

row	a	b	c	q_{mul}	q_{add}
0	a_0	b_0	c_0	1	0
1	a_1	b_1	c_1	0	1
2	a_2	b_2	c_2	1	0

- row は計算ステップを置く行である。
- a, b, c は wire 値を置く advice column であり、prover が witness や中間値から埋める。
- $q_{\text{mul}}, q_{\text{add}}$ は fixed column に置く selector であり、その行でどの gate を有効にするかを選ぶ。gate は各行で満たすべき式である。
- 公開入力 instance column に置く。別々のセルが同じ値であることは copy constraint で保証する。

Plonkish の gate を式で見る

各行で満たすべき gate は、selector を係数にした 1 本の式で書ける。

$$q_M ab + q_L a + q_R b + q_O c + q_C = 0$$

gate	q_M	q_L	q_R	q_O	q_C	得られる制約
mul	1	0	0	-1	0	$ab - c = 0$
add	0	1	1	-1	0	$a + b - c = 0$
const	0	1	0	0	$-k$	$a - k = 0$

selector の値を変えるだけで、同じ 1 本の式が行ごとに違う制約になる。たとえば mul 行では $q_M = 1$, $q_O = -1$ だけが立ち、 $ab - c = 0$ が残る。

列を多項式として扱う

行数を n 、評価領域を $H = \{1, \omega, \dots, \omega^{n-1}\}$ とする。列 a を、各行の値を評価値として持つ多項式 $A(X)$ で表す。

$$A(\omega^i) = a_i$$

値の列から多項式を作るこの補間は、多項式の準備で見た IFFT の仕事である。

$B, C, Q_M, \dots$ も同様に多項式になり、gate の式は 1 本の多項式にまとまる。

$$G(X) = Q_M AB + Q_L A + Q_R B + Q_O C + Q_C$$

すべての行で gate が成り立つことは、

$$G(\omega^i) = 0 \quad (i = 0, \dots, n-1)$$

つまり「 G が H 上で 0」と言い換えられる。多数の制約が多項式の等式に翻訳され、Schwartz–Zippel や commitment の道具が使えるようになる。

Plonkish で小例を配置する

$x^3 + x + 5 = y$ を 3 本の advice 列 a, b, c に置く。gate は 2 入力なので、足し算 $y = t_2 + x + 5$ は $t_3 = x + 5$ と $y = t_2 + t_3$ の 2 行に分ける。

row	a	b	c	gate
0	x	x	t_1	$a \cdot b = c$
1	t_1	x	t_2	$a \cdot b = c$
2	x	5	t_3	$a + b = c$
3	t_2	t_3	y	$a + b = c$

行ごとの gate はこれで揃った。ただし、row 0 の c と row 1 の a のように、行をまたいで「同じ値」を使っている箇所が残っている。

copy constraint: セルどうしを結ぶ

表のセルは、場所が違えば別の変数。row 0 で計算した t_1 を row 1 で使うには、「row 0 の c と row 1 の a は同じ値」だと制約で言う必要がある。これが copy constraint。

小例で結ぶセル

t_1 : row 0 の c と row 1 の a

x : row 0 の a, b , row 1 の b , row 2 の a

t_2 : row 1 の c と row 3 の a

t_3 : row 2 の c と row 3 の b

y : row 3 の c と公開入力 y

また、row 2 の $b = 5$ は fixed column や constant gate で固定する。

注意

表に同じ記号を書いただけでは、数学的には同じ値だと証明されない。equality も制約として入れる。

Lookup とは何か

lookup は、証明中で使った値が、あらかじめ固定した表に含まれることを示す制約である。複雑な計算を毎回 gate で展開する代わりに、「この行は許可表のどれかと一致する」と確認する。

証明中で使った値

$$v_0, v_1, \dots, v_{r-1}$$



固定された許可表

$$T = \{0, 1, \dots, 255\}$$

$$v_i \in T \quad (i = 0, \dots, r-1)$$

を短い lookup argument で証明する。

- 表 T は fixed column として回路側に置く。
- 問い合わせる値 v_i は advice column に置く。
- 実装では並べ替えや多重集合の一致を使うが、意味は table membership である。

値の組も同じ考え方で扱える。例えば $(\text{opcode}, a, b, c) \in T_{\text{op}}$ と書けば、1 行の命令が許可された命令表に載っていることを示す。

Lookup の応用例

lookup は、「小さな表で表せる関係」を回路の中で何度も使うときに効く。

用途	表に置くもの	何を簡潔に書けるか
range check	$T_{\text{byte}} = \{0, 1, \dots, 255\}$	$v \in T_{\text{byte}}$ と書くことで、byte 値であることを示す。bit ごとの制約を毎回並べない。
ハッシュ関数の部品	$T_{\text{sbox}} = \{(x, S(x))\}$	S-box や小さな非線形変換を、入力と出力の対応表として確認する。
zkVM の命令表	$T_{\text{op}} = \{(\text{op}, a, b, c)\}$	$(\text{op}, a, b, c) \in T_{\text{op}}$ として、opcode ごとの意味を表にまとめる。

lookup だけで計算全体の正しさが決まるわけではない。gate は行ごとの等式を確認し、copy constraint はセル同士の同一性を確認し、lookup は値が許可表に入っていることを確認する。

CCS: 制約形式を一般化する

ここまでで、R1CS・AIR・Plonkish という複数の書き方を見た。CCS(Customizable Constraint Systems) は、これらを 1 つの記法へまとめる一般化である。

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0$$

読み方: 行列 M_j が線形結合の束を作り、 S_i で選んだものどうしを掛け、係数 c_i を付けて足す。 S_i は同じ添字を複数回使える多重集合として扱う。

何が嬉しいか

R1CS、Plonkish 風の gate、高 degree gate を、「線形結合の積の和」という同じ記法で扱える。後で見る folding の土台になる。

CCS のデータ

有限体を $\mathbb{F}$ とする。CCS では次のデータを固定する。

- 行数 m と変数数 n 。
- 行列 $M_1, \dots, M_t \in \mathbb{F}^{m \times n}$ 。
- 項を指定する添字集合 $S_1, \dots, S_q$ 。厳密には多重集合として扱う。
- 係数 $c_1, \dots, c_q \in \mathbb{F}$ 。

公開入力 x 、witness w 、定数 1 を含めて

$$z = (w, x, 1) \in \mathbb{F}^n$$

と置く。順番は方式や文献ごとの約束であり、大事なものは必要な値をすべて z に含めることである。

CCS の式を成分ごとに読む

CCS の制約は

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0 \in \mathbb{F}^m$$

である。

Hadamard 積は成分ごとの積:

$$(u \circ v)_r = u_r v_r$$

成分ごとに書くと、各行 r について

$$\sum_{i=1}^q c_i \prod_{j \in S_i} (M_j z)_r = 0$$

を要求している。

R1CS は CCS の特別な場合

R1CS は

$$(Az) \circ (Bz) - Cz = 0$$

と書ける。

CCS で

$$M_1 = A, \quad M_2 = B, \quad M_3 = C$$

$$S_1 = \{1, 2\}, \quad S_2 = \{3\}, \quad c_1 = 1, \quad c_2 = -1$$

と置くと、

$$(M_1 z) \circ (M_2 z) - (M_3 z) = 0$$

となり、R1CS を含む。

Plonkish gate も CCS で見る

Plonkish の gate

$$q_{Mab} + q_L a + q_R b + q_O c + q_C = 0$$

も、行ごとに見れば線形結合の積の和。

$$(M_1 z) \circ (M_2 z) + (M_3 z) + (M_4 z) + (M_5 z) + (M_6 z) = 0$$

高 degree gate も同じ見方で扱える。例えば

$$a_i b_i c_i - d_i = 0$$

は

$$(M_1 z) \circ (M_2 z) \circ (M_3 z) - (M_4 z) = 0$$

と書ける。

CCS が folding で役立つ理由

folding scheme では、二つの証明対象を一つの証明対象へまとめる。そのとき、制約が共通の形で書かれていると扱いやすい。

$$\sum_i c_i \cdot \prod_{j \in S_i} \text{linear form}_{i,j}(z) = 0$$

分解できる操作

- 線形結合を作る。
- 同じ行どうしで掛ける。
- 係数を掛けて足す。

R1CS、Plonkish 風の gate、高 degree gate を共通インターフェースへ持ち込みやすい。

Arithmetization の比較

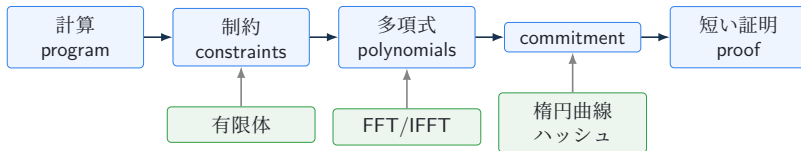
方式	主な表現	得意な計算	よく出る文脈	理解
R1CS	乗算制約の集合	算術回路	Groth16、Circom	回路を制約に分解する
AIR	実行トレースと遷移	VM、繰り返し計算	STARK	計算履歴が正しいか見る
Plonkish	gate と copy constraint	汎用回路、lookup	Plonk 系	表計算のように制約を書く
CCS	一般化された制約	folding-friendly な表現	Nova 系	複数方式をまとめる抽象化

最低限覚えること

arithmetization は、計算を有限体上の制約へ翻訳する工程である。方式ごとに「何を自然に表せるか」が違う。

全体像をもう一度

導入で見た地図に、今日学んだ道具を書き込める。



- 計算 → 制約: arithmetization の仕事。R1CS・AIR・Plonkish・CCS という流儀を見た。
- 制約 → 多項式: 列を補間して 1 本の等式へ。FFT/IFFT と Schwartz-Zippel が支える。
- 多項式 → commitment: KZG が多項式を群要素 1 個に固定する。土台は楕円曲線とペアリング。
- 対話 → 短い proof: Fiat-Shamir 変換でチャレンジを hash に置き換え、proof1 通にする。

一言で言うと

zkSNARK は、公開 statement に対して秘密 witness が存在することを、計算を制約と多項式へ翻訳し、commitment で短く固定して、短い proof として検証する仕組みである。

Further Reading

- RareSkills ZK Book
- MoonMath Manual
- Ethereum EIP-4844: blob transactions
- Kate polynomial commitments (Dankrad Feist)
- クラウドを支えるこれからの暗号技術
- 入門チュートリアル: Circom, Noir, Halo2 book



補足: 記法と用語

記法

記号	意味
x	公開入力。verifier にも見える値。
w	witness。statement を成り立たせる補助情報。
π	proof。verifier へ渡す短い証拠。
$R(x, w) = 1$	関係 R が、公開入力 x と witness w を受理すること。 statement は通常 $\exists w : R(x, w) = 1$ 。
$\mathbb{F}_p$	素数 p 個の元からなる有限体。
$\mathbb{F}_p^*$	$\mathbb{F}_p$ から 0 を除いた集合。掛け算について群になる集合。
$\langle g \rangle$	元 g が生成する巡回群。
g^a, aP	乗法群と楕円曲線の加法群で、秘密スカラーを掛けた公開値。
$E/k, e$	体 k 上の楕円曲線と、 $e : G_1 \times G_2 \rightarrow G_T$ のようなペアリング。

用語

用語	意味
statement	検証者が確認したい公開された主張。
proof / argument	statement が正しいこと、または witness を知っていることを示す証拠。witness の秘匿は zero-knowledge 性が担う。
transcript hash	protocol 名、statement、commitment などの公開ログから作る hash。
commitment	値や多項式を短い値に固定し、必要な部分だけ open する仕組み。
binding / hiding	commit 後に別の値へ開けない性質と、commitment から値が漏れない性質。
arithmetization	計算を有限体上の制約へ翻訳する工程。
R1CS / AIR / Plonkish / CCS	計算を制約として表すための代表的な表現形式。
離散対数問題	公開値から秘密スカラーを求める問題。
計算	Diffie-Hellman 型の値を計算する問題。
Diffie-Hellman 問題	
判定	Diffie-Hellman 型の値かどうかを判定する問題。
Diffie-Hellman 問題	
SRS	KZG などを使う Structured Reference String。公開パラメータ。